

LAPORAN PRAKTIKUM
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK
PERTEMUAN 3
CONTINUOUS DEPLOYMENT UNTUK LARAVEL



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
Kelas : PL5A1
Dosen Pengampu : Galih Malela Damaraji

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Continuous Deployment dan Perbedaannya dengan Continuous Integration	4
2.2 Pipeline Bertahap dengan needs :	4
2.3 Environment dan Required Reviewer	5
2.4 Urutan Langkah Deploy Laravel	5
2.5 Artefak Antar Job	5
Hasil dan Pembahasan	7
3.1 Alat dan Bahan	7
3.2 Penyiapan Lingkungan Lokal	7
3.3 Workflow Empat Job	8
3.3.1 Menjaga Agar Production Hanya Jalan dari main	9
3.3.2 Required Reviewer pada Environment Production	9
3.4 Skrip Deploy di Sisi Server	10
3.5 Bukti Pipeline Berhenti Saat Pengujian Gagal	11
3.6 Alur Pull Request dan Hasil Akhir	12
3.7 Kendala yang Ditemui	13
Kesimpulan	14
Daftar Pustaka	15

Tujuan Praktikum

Sesuai arahan pada pertemuan ketiga dan deskripsi tugas di Google Classroom *Konstruksi dan Evolusi Perangkat Lunak 2026*, tujuan praktikum ini adalah:

1. Mahasiswa mampu menyusun pipeline empat tahap (build, test, staging, production) yang dirangkai dengan *needs* :.
2. Mahasiswa mampu menjaga agar tahap deploy hanya berjalan dari branch main dan menunggu persetujuan manusia lewat *environment* dengan *required reviewer*.
3. Mahasiswa mampu menuliskan urutan langkah deploy Laravel di server sebagai skrip `deploy.sh` yang berhenti saat ada perintah gagal.
4. Mahasiswa mampu membuktikan bahwa pipeline benar-benar menghentikan tahap berikutnya ketika satu pengujian sengaja digagalkan.

Dasar Teori

2.1 Continuous Deployment dan Perbedaannya dengan Continuous Integration

Continuous Integration (CI) memverifikasi setiap perubahan secara otomatis, sedangkan Continuous Deployment (CD) melanjutkan langkah itu sampai perubahan benar-benar terpasang di server [1, 2]. Pertemuan sebelumnya berhenti di tahap CI: kode diuji dan gaya kodenya diperiksa, lalu selesai. Pertemuan ini menambah tahap setelahnya, yaitu *staging* dan *production*, sehingga pipeline menyerupai alur rilis yang sebenarnya.

2.2 Pipeline Bertahap dengan *needs* :

GitHub Actions menjalankan setiap job di mesin yang bersih, sehingga tiap job harus menyiapkan ulang apa yang dibutuhkannya [3]. Kata kunci *needs* : membuat sebuah job menunggu job lain selesai lebih dulu [4]. Kalau job yang ditunggu gagal, job setelahnya tidak dijalankan sama sekali dan statusnya menjadi *skipped*. Sifat inilah yang dipakai untuk menghentikan deploy ketika pengujian gagal.

```
hafidz@laptop:~/Project/evolusi-pl-535493$ gh run view 35199310754 --json jobs --jq '.jobs[]|"{(.name): \(.conclusion)}"'  
Working...  
Working...  
Build: success  
Test: success  
Staging: success  
Production: success
```

Gambar 1: Keempat job pipeline (Build, Test, Staging, Production) hijau pada satu run.

2.3 Environment dan Required Reviewer

Environment pada GitHub Actions adalah wadah berisi variabel dan aturan perlindungan untuk satu tujuan deploy [5]. Ketika sebuah job memakai `environment: production` dan environment tersebut punya *required reviewer*, job berhenti di status *waiting* sampai ada orang yang menyetujui. Cara ini membuat deploy ke produksi tidak pernah berjalan sendiri tanpa sepengetahuan pemilik proyek.

2.4 Urutan Langkah Deploy Laravel

Deploy aplikasi Laravel di server mengikuti urutan yang tidak boleh ditukar [7]. Aplikasi ditutup dulu dengan `php artisan down`, kode terbaru ditarik, dependensi produksi dipasang tanpa paket pengembangan, migrasi basis data dijalankan, cache dibangun ulang, pekerja antrian dimuat ulang, lalu aplikasi dibuka kembali dengan `php artisan up`. Baris `set -e` di awal skrip penting karena tanpa itu skrip akan terus berjalan walaupun satu perintah gagal, dan aplikasi bisa terbuka dalam keadaan rusak [8].

2.5 Artefak Antar Job

Karena setiap job mulai dari mesin kosong, hasil pekerjaan satu job perlu diteruskan ke job berikutnya. `actions/upload-artifact` dan `actions/download-artifact` dipakai untuk memindahkan berkas antar job [6]. Pada praktikum ini direktori `vendor` hasil

composer install di job build diunggah sebagai artefak, lalu diunduh oleh job test, sehingga pemasangan dependensi cukup dilakukan sekali.

Hasil dan Pembahasan

Pada bab ini saya menjelaskan pekerjaan untuk Tugas 2 sesuai instruksi di Classroom. Saya melanjutkan repository `evolusi-pl-535493` dari minggu sebelumnya, tidak membuat repository baru. Semua langkah saya verifikasi lewat GitHub Actions dan eksekusi lokal.

3.1 Alat dan Bahan

Alat dan bahan yang digunakan:

- Laptop dengan OS Linux, akses internet, dan akun GitHub `hafidzrizqullahprasetya`.
- Git, GitHub CLI (`gh`), PHP 8.5.10, Composer 2.10.3, Laravel 13, PHPUnit, dan Laravel Pint.
- Repository: `hafidzrizqullahprasetya/evolusi-pl-535493` (publik) dengan aplikasi Laravel berisi kalkulator IP semester dari pertemuan pertama.
- Google Classroom: kelas *Konstruksi dan Evolusi Perangkat Lunak 2026* [875340560422], tugas *Pengumpulan Tugas 2* dengan tenggat 26 September 2026.

3.2 Penyiapan Lingkungan Lokal

Pada pertemuan pertama pengujian hanya berjalan di GitHub Actions karena laptop belum punya PHP. Untuk praktikum ini saya memasang PHP 8.5 lewat PPA `ondrej/php` beserta Composer, supaya perintah `php artisan test` bisa dijalankan di laptop seperti yang diminta pada langkah tugas. Saya memilih versi 8.5 agar sama dengan versi yang dipakai runner di workflow. Setelah pemasangan, pengujian dan pemeriksaan gaya kode dijalankan langsung di laptop:

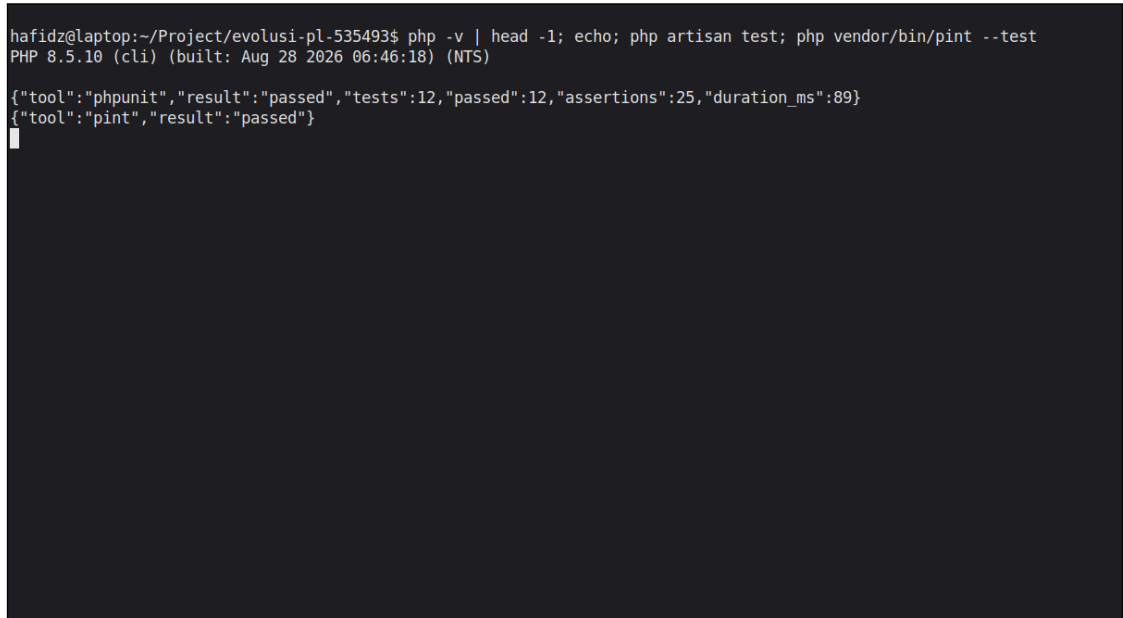
```
1 php -v
2 # PHP 8.5.10 (cli) (built: Aug 28 2026 06:46:18) (NTS)
3
```

```

4 php artisan test
5 # 12 tests, 12 passed, 25 assertions
6
7 php vendor/bin/pint --test
8 # passed

```

Listing .1: Verifikasi lokal setelah PHP dipasang.



```

hafidz@laptop:~/Project/evolusi-pl-535493$ php -v | head -1; echo; php artisan test; php vendor/bin/pint --test
PHP 8.5.10 (cli) (built: Aug 28 2026 06:46:18) (NTS)

{"tool":"phpunit","result":"passed","tests":12,"passed":12,"assertions":25,"duration_ms":89}
{"tool":"pint","result":"passed"}

```

Gambar 1: Hasil pengujian dan pemeriksaan gaya kode di laptop setelah PHP 8.5 dipasang.

3.3 Workflow Empat Job

Workflow `.github/workflows/ci.yml` saya perluas dari dua job menjadi empat job berurutan. Job build memasang dependensi dan mengunggah direktori vendor sebagai artefak. Job test mengunduh artefak itu, menyiapkan berkas `.env`, menjalankan migrasi, lalu menjalankan `php artisan test` dan pemeriksaan `pint`. Job staging dan production dirangkai dengan `needs`: setelahnya.

```

1 jobs:
2   build:
3     name: Build
4     ...
5   test:
6     name: Test
7     needs: build
8     ...
9   staging:
10    name: Staging
11    needs: test
12    ...

```

```

13 production:
14   name: Production
15   needs: staging
16   if: github.ref == 'refs/heads/main' && github.event_name == 'push'
17   environment: production
18   ...

```

Listing .2: Kerangka empat job pada ci.yml.

3.3.1 Menjaga Agar Production Hanya Jalan dari main

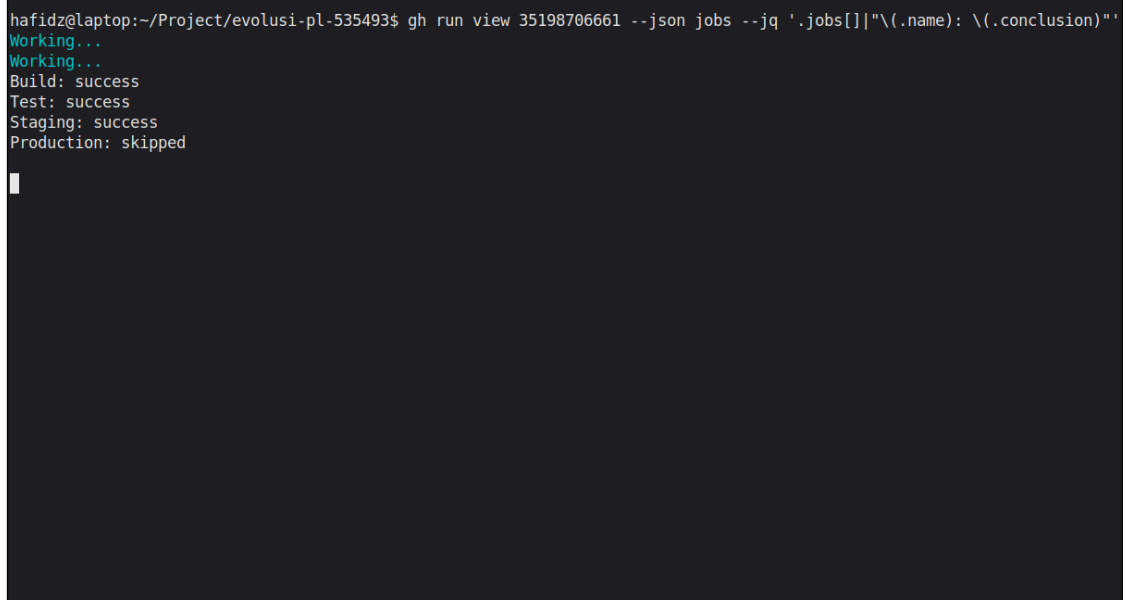
Job production diberi syarat if: `github.ref == 'refs/heads/main'` dan hanya bereaksi pada kejadian push. Karena itu, ketika kode di-push ke branch fitur, job ini dilewati walaupun job build, test, dan staging tetap berjalan. Saya membuktikannya dengan push ke branch `feature/pipeline-deploy`:

```

1 gh run view 35198706661 --json jobs --jq '.jobs[] | "\(.name): \(.conclusion)'"
2 # Build: success
3 # Test: success
4 # Staging: success
5 # Production: skipped

```

Listing .3: Job production terlewat pada push ke branch fitur.



```

hafidz@laptop:~/Project/evolusi-pl-535493$ gh run view 35198706661 --json jobs --jq '.jobs[] | "\(.name): \(.conclusion)'"
Working...
Working...
Build: success
Test: success
Staging: success
Production: skipped

```

Gambar 2: Push ke branch fitur menjalankan build, test, dan staging, sementara production terlewat.

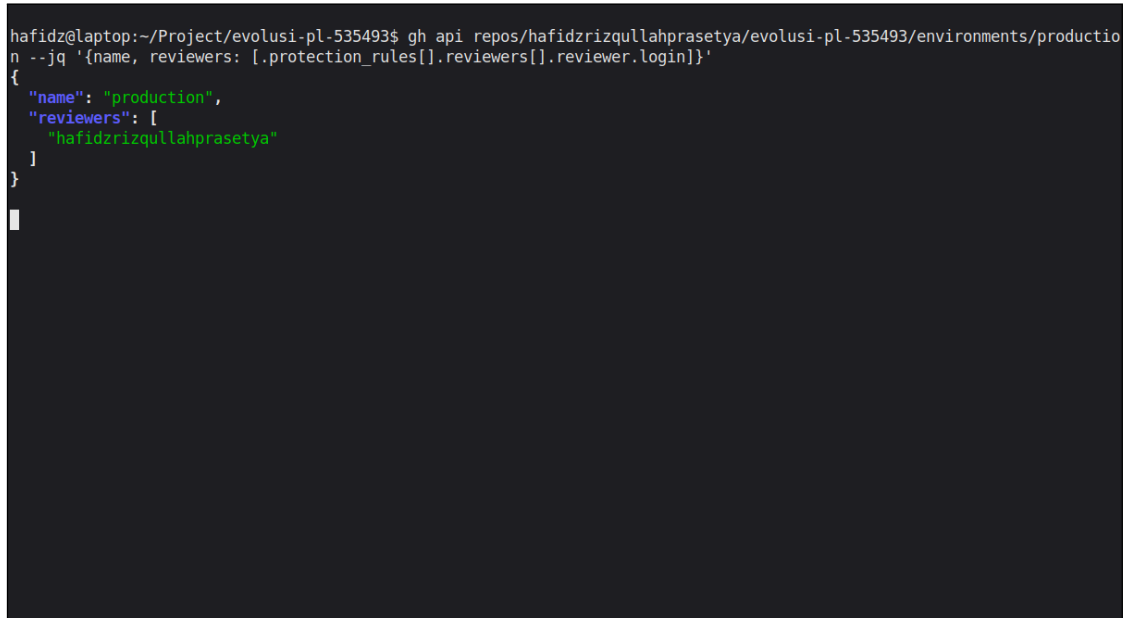
3.3.2 Required Reviewer pada Environment Production

Supaya deploy ke produksi menunggu persetujuan, saya membuat environment bernama `production` dan menambahkan diri sendiri sebagai *required reviewer*. Verifikasi lewat API

menunjukkan environment tersebut beserta reviewernya:

```
1 gh api repos/.../environments/production \
2   --jq '{name, reviewers: [.protection_rules[].reviewers[].reviewer.login]}'
3 # {"name":"production","reviewers":["hafidzrizqullahprasetya"]}
```

Listing .4: Verifikasi environment production dan reviewernya.



```
hafidz@laptop:~/Project/evolusi-pl-535493$ gh api repos/hafidzrizqullahprasetya/evolusi-pl-535493/environments/production --jq '{name, reviewers: [.protection_rules[].reviewers[].reviewer.login]}'
{
  "name": "production",
  "reviewers": [
    "hafidzrizqullahprasetya"
  ]
}
```

Gambar 3: Environment production dengan required reviewer yang terdaftar.

Ketika perubahan akhirnya masuk ke main, job production berhenti di status *waiting* sampai saya menyetujuinya. Setelah disetujui, barulah job itu berjalan sampai selesai. Ini memperlihatkan bahwa tahap deploy benar-benar menunggu persetujuan manusia, bukan langsung jalan.

3.4 Skrip Deploy di Sisi Server

Langkah deploy di server saya tulis sebagai berkas `deploy.sh` di root repository. Isinya urutan perintah dari `artisan down` sampai `artisan up`, dengan `set -e` di baris awal agar skrip berhenti begitu ada perintah yang gagal.

```
1 #!/usr/bin/env bash
2 set -e
3
4 cd /var/www/aplikasi
5
6 php artisan down --retry=60
7 git pull origin main
8 composer install --no-dev --optimize-autoloader
9 php artisan migrate --force
```

```

10 php artisan config:cache
11 php artisan route:cache
12 php artisan view:cache
13 php artisan queue:restart
14 php artisan up

```

Listing .5: Isi deploy.sh.

Sesuai instruksi tugas, tahap deploy belum benar-benar dikirim ke server. Ketujuh langkah itu dituliskan sebagai echo di dalam job production supaya urutannya terlihat di log. Hasilnya tampak pada Gambar 4.

```

hafidz@laptop:~/Project/evolusi-pl-535493$ gh run view --job=105130249778 --log | grep -E 'Z [0-9]\. '
Production    Deploy ke produksi      2026-09-17T08:24:21.2253650Z 1. php artisan down --retry=60
Production    Deploy ke produksi      2026-09-17T08:24:21.2255994Z 2. git pull origin main
Production    Deploy ke produksi      2026-09-17T08:24:21.2257380Z 3. composer install --no-dev --optimize-autoloader
Production    Deploy ke produksi      2026-09-17T08:24:21.2258493Z 4. php artisan migrate --force
Production    Deploy ke produksi      2026-09-17T08:24:21.2259816Z 5. php artisan config:cache && php artisan route:ca
che && php artisan view:cache
Production    Deploy ke produksi      2026-09-17T08:24:21.2260940Z 6. php artisan queue:restart
Production    Deploy ke produksi      2026-09-17T08:24:21.2261807Z 7. php artisan up

```

Gambar 4: Log job Production yang menampilkan ketujuh langkah deploy secara berurutan.

3.5 Bukti Pipeline Berhenti Saat Pengujian Gagal

Instruksi tugas meminta saya sengaja menggagalkan satu pengujian untuk membuktikan pipeline berhenti. Saya mengubah nilai harapan pada satu kasus uji di tests/Unit/IpCalculatorTest.php dari 3.5 menjadi 9.9, lalu push. Job test gagal, dan job staging serta production otomatis terlewat karena needs: tidak terpenuhi.

```

1 gh run view 35198536212 --json jobs --jq '.jobs[] | "{.name}: {:.conclusion}"'
2 # Build: success
3 # Test: failure
4 # Staging: skipped
5 # Production: skipped
6
7 # Failed asserting that 3.5 matches expected 9.9.
8 # Tests: 1 failed, 11 passed (25 assertions)

```

Listing .6: Pipeline merah pada satu pengujian yang sengaja digagalkan.

```
hafidz@laptop:~/Project/evolusi-pl-535493$ gh run view 35198536212 --json jobs --jq '.jobs[] | "{(.name): \(.conclusion)}"'
; echo; gh run view 35198536212 --log-failed | sed -r 's/\^[^[[0-9;]*m//g' | grep -E 'FAIL|Failed asserting|Tests:' | s
ed -E 's/^.*/Z //'
Working...
Working...
Build: success
Test: failure
Staging: skipped
Production: skipped

FAIL Tests\Unit\IpCalculatorTest
FAILED Tests\Unit\IpCalculatorTest > hitung i p menghitung rata rata ber...
Failed asserting that 3.5 matches expected 9.9.
Tests: 1 failed, 11 passed (25 assertions)
```

Gambar 5: Satu pengujian yang digagalkan menghentikan pipeline: test gagal, staging dan production terlewat.

Setelah bukti itu tersimpan, saya mengembalikan nilai harapan ke 3.5 dan push lagi. Pipeline kembali hijau pada commit berikutnya.

3.6 Alur Pull Request dan Hasil Akhir

Semua perubahan tetap melalui Pull Request sesuai aturan branch protection dari pertemuan pertama. Saya membuat branch `feature/pipeline-deploy` dari `dev`, membuka PR ke `dev`, lalu meneruskan `dev` ke `main`. Karena branch protection memakai status check wajib, saya juga memperbarui daftar check yang diwajibkan agar cocok dengan nama job baru (`Build` dan `Test`). Pada run terakhir di `main`, keempat job hijau:

```
1 gh run view 35199310754 --json jobs --jq '.jobs[] | "{(.name): \(.conclusion)}"'
2 # Build: success
3 # Test: success
4 # Staging: success
5 # Production: success
```

Listing .7: Keempat job hijau pada run di branch `main`.

```
hafidz@laptop:~/Project/evolusi-pl-535493$ gh run view 35199310754 --json jobs --jq '.jobs[]|\"(.name): \"(.conclusion)\"'
Working...
Working...
Build: success
Test: success
Staging: success
Production: success
```

Gambar 6: Keempat job hijau pada run di main setelah PR digabungkan dan deploy produksi disetujui.

3.7 Kendala yang Ditemui

Ada beberapa kendala yang saya temui. Laptop belum punya PHP, jadi pengujian lokal tidak bisa dijalankan sebelum PHP 8.5 dipasang; setelah terpasang, hasilnya cocok dengan hasil di runner. Kendala lain muncul saat direktori vendor dipindahkan antar job lewat artefak: izin eksekusi berkas `vendor/bin/pint` hilang, sehingga job test sempat gagal dengan pesan *Permission denied*. Saya memperbaikinya dengan memanggil `php vendor/bin/pint` alih-alih `./vendor/bin/pint`. Mengubah nama job juga sempat menghambat karena status check wajib pada branch protection tidak lagi cocok dengan nama job yang baru, jadi daftar check perlu disesuaikan dulu sebelum PR bisa digabungkan.

Kesimpulan

Berdasarkan praktikum pada Pertemuan 3, dapat disimpulkan beberapa hal:

1. Workflow berhasil diperluas menjadi empat job (`build`, `test`, `staging`, `production`) yang dirangkai dengan `needs` :, dan direktori `vendor` dipindahkan antar job lewat artefak sehingga pemasangan dependensi cukup sekali.
2. Job `production` terbukti hanya berjalan dari branch `main`; push ke branch fitur membuat job itu melewati sementara `build`, `test`, dan `staging` tetap berjalan.
3. Environment `production` dengan required reviewer membuat deploy menunggu persetujuan manusia, dan setelah disetujui barulah job berjalan sampai selesai.
4. Ketujuh langkah deploy Laravel dituliskan pada `deploy.sh` dengan `set -e`, lalu ditampilkan sebagai `echo` di log job `production` sesuai urutan.
5. Pipeline terbukti berhenti ketika satu pengujian sengaja digagalkan: `job test` gagal dan job setelahnya melewati, kemudian kembali hijau setelah kesalahan diperbaiki.

Dengan ini pipeline repository `evolusi-pl-535493` sudah berjalan dari pengujian sampai deploy, dengan penjaga branch dan persetujuan pada tahap produksi.

Daftar Pustaka

- [1] M. Fowler, “Continuous Integration.” [Online]. Tersedia: <https://martinfowler.com/articles/continuousIntegration.html>.
- [2] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [3] GitHub Docs, “Understanding GitHub Actions.” [Online]. Tersedia: <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>.
- [4] GitHub Docs, “Using jobs.” [Online]. Tersedia: <https://docs.github.com/en/actions/using-jobs>.
- [5] GitHub Docs, “Using environments for deployment.” [Online]. Tersedia: <https://docs.github.com/en/actions/deployment/targeting-different-environments/using-environments-for-deployment>.
- [6] GitHub Docs, “Storing and sharing data from a workflow.” [Online]. Tersedia: <https://docs.github.com/en/actions/using-workflows/storing-workflow-data-as-artifacts>.
- [7] Laravel, “Laravel Documentation.” [Online]. Tersedia: <https://laravel.com/docs>.
- [8] M. Stauffer, *Laravel: Up & Running*, 3rd ed. O’Reilly Media, 2023.
- [9] PHPUnit, “PHPUnit Documentation.” [Online]. Tersedia: <https://docs.phpunit.de/>.
- [10] Laravel, “Laravel Pint.” [Online]. Tersedia: <https://laravel.com/docs/pint>.